

QMOOD-Based Software Quality Evolution Analysis and LLM-Assisted Assessment: A Case Study on picoccli

Levent Bayraktar and Esra Şevval Bayraktar

Abstract—This paper presents a QMOOD-based software quality evolution analysis of the open-source Java project *picoccli*. Ten versions from v4.0.4 to v4.7.7 were analyzed. Object-oriented metrics were extracted using CK Tool, transformed into QMOOD design properties, normalized with respect to the first analyzed version, and then used to compute six quality attributes and the Total Quality Index (TQI). The results show that the number of classes increased from 192 to 219 and the number of methods increased from 1517 to 1848. TQI increased slightly from 4.0100 to 4.1015, indicating a limited positive overall trend. However, this total score hides important trade-offs: reusability, flexibility, functionality, and effectiveness improved, while understandability and extendibility decreased. In addition, a class-level risk analysis for v4.7.7 shows that risk is concentrated around `picoccli.CommandLine` and its inner classes. The study also compares Gemini, DeepSeek, and Claude outputs using two prompts and a six-criterion evaluation rubric. The LLM results show that language models can support metric interpretation, but they must be critically checked because they may overstate design smells or infer architectural problems not directly supported by the measured data.

Index Terms—QMOOD, CK metrics, object-oriented metrics, software quality, software evolution, maintainability, technical debt, large language models, *picoccli*.

I. INTRODUCTION AND PROBLEM STATEMENT

Software systems evolve continuously due to new requirements, bug fixes, refactoring decisions, and changes in their usage contexts. During this evolution, functionality may increase, but structural complexity, coupling, and maintenance effort may also increase. Therefore, software quality should not be evaluated only through subjective observations; it should also be supported by repeatable, metric-based analysis.

The main problem addressed in this work is the following: *How does the design quality of an object-oriented open-source software project change across versions, and how reliably can large language models interpret this change when they are given metric-based evidence?* To answer this question, the Java-based command-line interface library *picoccli* was selected as the target system. The analyzed versions are v4.0.4, v4.1.0, v4.1.4, v4.2.0, v4.3.0, v4.3.2, v4.4.0, v4.5.2, v4.6.3, and v4.7.7.

Instructor: Hakan Gündüz. This report was prepared for the Software Architecture course. Project repository: <https://github.com/esseucar/qmood-picoccli-quality-analysis>.

The study has four research questions:

- RQ1: How do object-oriented metrics such as WMC, RFC, CBO, DIT, and LCOM change across the selected *picoccli* versions?
- RQ2: How do QMOOD quality attributes and TQI reflect this change?
- RQ3: Do complexity, cohesion, and coupling metrics indicate architectural degradation or technical debt risk?
- RQ4: How accurately do LLMs interpret the metric-based quality results, and where do they produce unsupported claims?

The contribution of this study is threefold. First, it provides a reproducible metric extraction and QMOOD computation pipeline for ten versions of *picoccli*. Second, it interprets software evolution through both version-level trends and class-level risk analysis. Third, it critically evaluates LLM outputs using a six-criterion rubric and two different prompt designs. All project artifacts are stored in the GitHub repository: <https://github.com/esseucar/qmood-picoccli-quality-analysis>.

II. RELATED WORK

A. Object-Oriented Software Metrics

Object-oriented software metrics are widely used to evaluate structural properties such as complexity, coupling, inheritance, and cohesion. The CK metric suite proposed by Chidamber and Kemerer is one of the most commonly used suites for object-oriented design measurement [1]. In this suite, WMC represents class-level method complexity, DIT represents inheritance depth, CBO represents coupling between objects, RFC represents the response set of a class, and LCOM represents lack of cohesion. Li and Henry also showed that object-oriented metrics can be related to maintainability prediction [2].

B. QMOOD Quality Model

QMOOD is a hierarchical model for object-oriented design quality assessment [3]. It maps low-level design metrics to design properties and then to high-level quality attributes. The six high-level QMOOD attributes are reusability, flexibility, understandability, functionality, extendibility, and effectiveness. Since QMOOD combines multiple design properties into quality attributes, it is suitable for studying quality evolution over multiple versions.

C. Software Evolution and Technical Debt

Lehman's laws of software evolution emphasize that software systems must change continuously and that increasing complexity must be controlled during evolution [4]. Technical debt is also related to design decisions that make short-term progress easier but increase future maintenance cost [5]. Refactoring is a common response to such structural problems, especially when high complexity, low cohesion, or excessive coupling is observed [6].

D. LLM-Assisted Software Engineering Analysis

Large language models are increasingly used in software engineering tasks such as code explanation, defect localization, refactoring suggestion, and requirements analysis. Recent surveys report that LLMs can be useful assistants for software engineering, but they also emphasize risks such as hallucination, overconfident reasoning, and lack of grounding in concrete artifacts [7], [8]. For this reason, LLM-based software quality assessment should be treated as a supporting qualitative analysis rather than a replacement for metric-based evidence.

III. PROPOSED METHODOLOGY

A. Target System and Version Selection

The target system is *picocli*, an open-source Java library for building command-line applications [10]. It was selected because it is object-oriented, publicly available, versioned through Git tags, and analyzable with static metric tools. The ten selected versions cover the evolution from v4.0.4 to v4.7.7.

B. Metric Extraction Pipeline

The source code of each selected version was extracted from the corresponding Git tag. For each version, only the `src/main/java` directory was analyzed. CK Tool was used to produce `class.csv` and `method.csv` files for each version [9]. Python scripts were then used to create version-level summaries, compute QMOOD scores, generate charts, and extract risky classes for the final version.

The main repository structure contains raw metrics, processed metrics, QMOOD scores, figures, scripts, LLM prompts, LLM outputs, and the report. The main scripts are `qmood_calculation.py`, `generate_figures.py`, and `extract_v477_risky_classes.py`.

C. Mapping CK Metrics to QMOOD Properties

CK Tool does not directly produce every original QMOOD metric. Therefore, several QMOOD design properties were represented using proxy metrics derived from CK outputs. Table I shows the mapping used in this project.

All design properties were normalized with respect to v4.0.4. The QMOOD quality attributes were calculated using the following formulas:

TABLE I
MAPPING FROM CK-BASED MEASUREMENTS TO QMOOD DESIGN PROPERTIES

QMOOD property	Metric/proxy	Meaning
Design Size	<code>class_count</code>	Total number of classes
Hierarchies	<code>classes_with_children</code>	Classes having child classes
Abstraction	<code>avg_dit</code>	Average DIT proxy
Encapsulation	DAM	Private/protected field ratio
Coupling	<code>avg_cbo</code>	Average CBO
Cohesion	$1/(1+\text{avg_lcom})$	Inverse LCOM proxy
Composition	<code>total_fields_qty</code>	Field-count proxy
Inheritance	<code>avg_dit</code>	Inheritance-depth proxy
Polymorphism	<code>classes_with_children</code>	Inheritance-based proxy
Messaging	<code>public_methods_qty</code>	Public method count
Complexity	<code>avg_wmc</code>	Average WMC

$$\begin{aligned} Reusability = & 0.5DS - 0.25CPL + 0.25COH \\ & + 0.5MSG \end{aligned} \quad (1)$$

$$\begin{aligned} Flexibility = & 0.25ENC - 0.25CPL + 0.5CMP \\ & + 0.5POL \end{aligned} \quad (2)$$

$$\begin{aligned} Understandability = & -0.33DS - 0.33ABS + 0.33ENC \\ & - 0.33CPL + 0.33COH - 0.33POL \\ & - 0.33COM \end{aligned} \quad (3)$$

$$\begin{aligned} Functionality = & 0.22DS + 0.22HIE + 0.12COH \\ & + 0.22POL + 0.22MSG \end{aligned} \quad (4)$$

$$\begin{aligned} Extendibility = & 0.5ABS - 0.5CPL + 0.5INH \\ & + 0.5POL \end{aligned} \quad (5)$$

$$\begin{aligned} Effectiveness = & 0.2ABS + 0.2ENC + 0.2CMP \\ & + 0.2INH + 0.2POL \end{aligned} \quad (6)$$

The Total Quality Index was computed as follows:

$$\begin{aligned} TQI = & Reusability + Flexibility + Understandability \\ & + Functionality + Extendibility + Effectiveness. \end{aligned} \quad (7)$$

D. Prompt Engineering Strategy

Two prompt designs were used to evaluate LLM behavior. Prompt 1 was a detailed trend-analysis prompt. It provided metric tables, QMOOD scores, and explicit instructions to discuss quality evolution, maintainability, technical debt, architectural risks, and refactoring recommendations. Prompt 2 was a shorter and more structured prompt. It forced the models to summarize the trend, identify risks, and compare quality attributes in a more constrained format.

The prompts were designed with three goals. First, all models received the same quantitative evidence to make the comparison fair. Second, the prompts explicitly instructed the models to avoid unsupported claims and to distinguish metric-based findings from assumptions. Third, the use of two prompts made it possible to observe whether model behavior changed when the task was open-ended versus structured. Full prompt files are included in the project repository under the `prompts/` directory. Table II summarizes the prompt design.

TABLE II
PROMPT ENGINEERING DESIGN USED IN THE LLM EVALUATION

Prompt	Purpose	Main constraints
Prompt 1	Detailed interpretation of CK metrics, QMOOD attributes, TQI trend, maintainability, technical debt, and refactoring priorities.	Use only the supplied metrics; explain trade-offs; avoid class-level design-smell claims unless supported; discuss limitations of normalized QMOOD scores.
Prompt 2	Structured comparison prompt for concise LLM outputs and easier model-to-model comparison.	Provide trend summary, risks, refactoring suggestions, and limitations in a fixed structure; do not treat LLM inference as direct evidence.

E. LLM Evaluation Rubric

The LLM outputs were evaluated using six criteria: metric grounding, consistency, technical correctness, refactoring quality, hallucination risk control, and version-trend understanding. Each criterion was scored between 1 and 5. This produces a maximum score of 30 points. The consistency criterion was included to ensure alignment with the project rubric and to check whether a model contradicted itself across its explanation.

F. Class-Level Risk Score

Version averages may hide risky outlier classes. Therefore, a class-level risk score was computed for v4.7.7. Each metric was normalized and then combined using a weighted sum. The risk score is defined as follows:

$$Risk(c) = 0.35LCOM_n(c) + 0.25WMC_n(c) + 0.20RFC_n(c) + 0.15CBO_n(c) + 0.05DIT_n(c). \quad (8)$$

The weights emphasize cohesion risk and complexity because high LCOM, WMC, and RFC values are especially relevant to maintainability and testing effort.

IV. ANALYSIS PROCESS

The analysis process consisted of five steps. First, the ten selected versions were extracted from Git tags. Second, CK Tool was executed on each version to generate class-level and method-level CSV files. Third, version summaries were produced from these raw files. Fourth, QMOOD design properties and quality attributes were computed using the mapping in Table I and Equations 1–7. Fifth, LLM outputs were generated using Prompt 1 and Prompt 2 and evaluated with the six-criterion rubric.

To ensure reproducibility, the raw CK outputs for all ten versions were retained in `data/raw_metrics/`. The processed summaries were stored under `data/processed_metrics/`, and the final QMOOD scores were stored under `data/qmood_scores/`. The figure generation and risk analysis scripts were also included in the repository.

V. RESULTS

A. Version-Level Metric Evolution

Table III summarizes the main version-level metrics. The number of classes increased from 192 to 219, and the number of methods increased from 1517 to 1848.

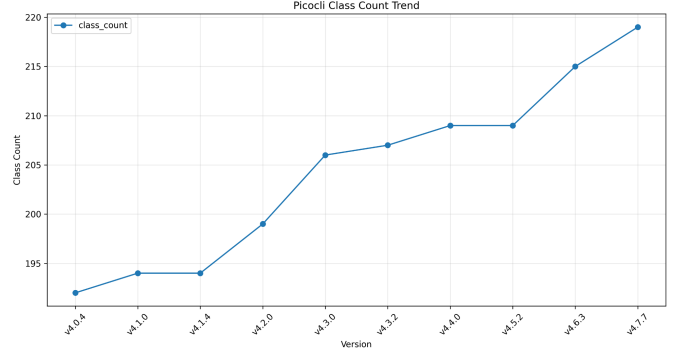


Fig. 1. Class count trend across the analyzed picocli versions.

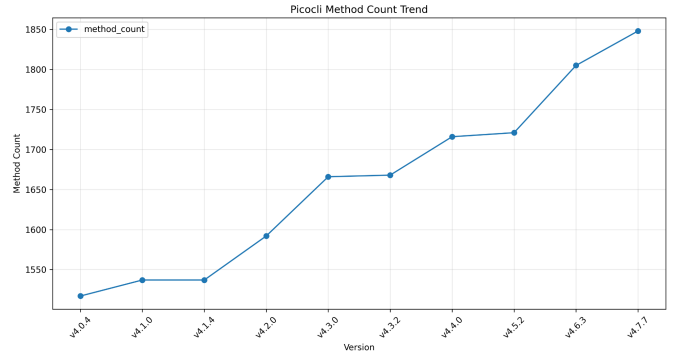


Fig. 2. Method count trend across the analyzed picocli versions.

Average WMC increased from 17.4048 to 19.5616, and average RFC increased from 14.2969 to 15.7763. These results indicate that the system grew and that behavioral complexity also increased. Average LCOM increased from 157.80 to 214.35, which is a stronger warning sign for cohesion and maintainability. CBO increased only slightly from 4.9219 to 5.0411, indicating that coupling changed in a limited but observable way. DAM remained stable and slightly increased from 0.7739 to 0.7790, suggesting that field encapsulation did not deteriorate.

Figures 1 and 2 show that the system grew structurally. Figure 3 shows the simultaneous increase in WMC and RFC, indicating growing implementation and testing complexity. Figure 4 shows that coupling increased only slightly. Figure 5 shows the most visible negative trend: LCOM increased strongly, which supports the interpretation that some classes accumulated multiple responsibilities. Figure 6 shows that average inheritance depth did not increase; therefore, the main quality risk is not deeper

TABLE III
VERSION-LEVEL METRIC SUMMARY

Version	Classes	Methods	Avg CBO	Avg WMC	Avg DIT	Avg RFC	Avg LCOM	DAM
v4.0.4	192	1517	4.9219	17.4048	1.8125	14.2969	157.80	0.7739
v4.1.0	194	1537	4.9485	17.6031	1.8041	14.4485	160.19	0.7737
v4.1.4	194	1537	4.9558	17.6959	1.8041	14.5052	160.93	0.7737
v4.2.0	199	1592	4.9950	18.0000	1.7839	14.7437	165.29	0.7723
v4.3.0	206	1666	4.9854	18.5097	1.7700	15.0971	173.64	0.7743
v4.3.2	207	1668	4.9807	18.4203	1.7826	15.0338	173.04	0.7727
v4.4.0	209	1716	4.9569	18.7895	1.7943	15.2105	185.55	0.7758
v4.5.2	209	1721	4.9809	18.8325	1.7943	15.2919	185.23	0.7758
v4.6.3	215	1805	5.0093	19.2419	1.7721	15.5163	209.29	0.7737
v4.7.7	219	1848	5.0411	19.5616	1.7580	15.7763	214.35	0.7790

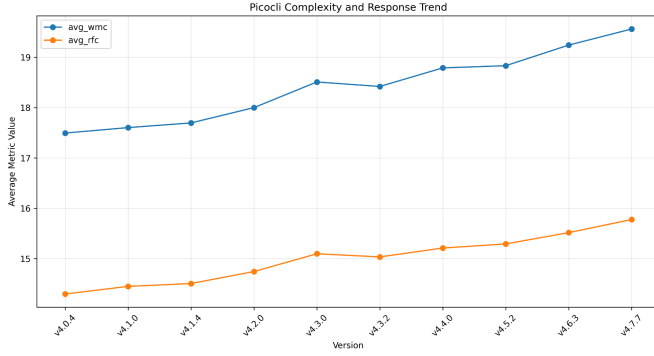


Fig. 3. Average WMC and RFC trend.

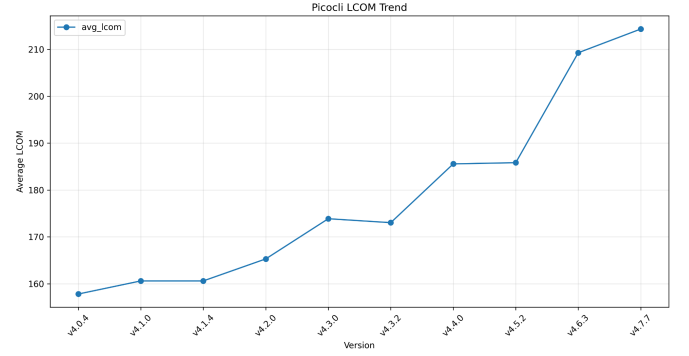


Fig. 5. Average LCOM trend.

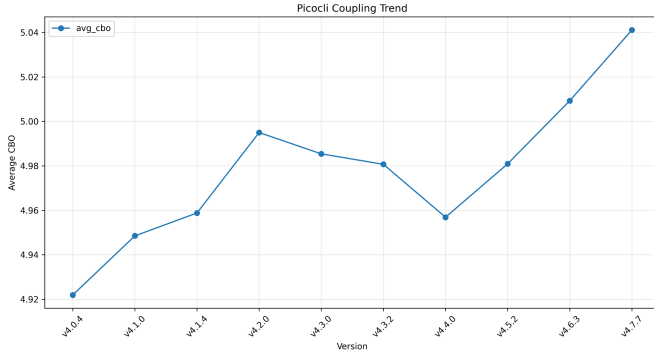


Fig. 4. Average CBO trend. The y-axis is intentionally narrowed to make the small numerical variation visible; the increase should not be visually overinterpreted.

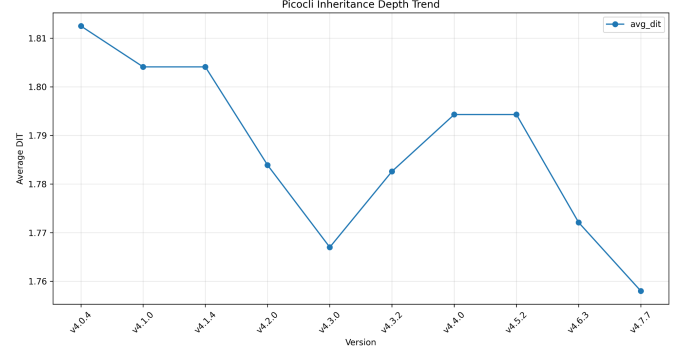


Fig. 6. Average DIT trend.

inheritance but complexity and cohesion degradation.

B. QMOOD Quality Results

Table IV shows the computed QMOOD quality attributes and TQI values. TQI increased from 4.0100 in v4.0.4 to 4.1015 in v4.7.7, corresponding to an approximate 2.28% increase. However, the total quality score must be interpreted carefully. Reusability, flexibility, functionality, and effectiveness improved, while understandability and extendibility decreased.

Figures 7 and 8 confirm that the overall TQI improvement is positive but limited. Figure 9 shows that understandability decreases as the project grows. Figure 10

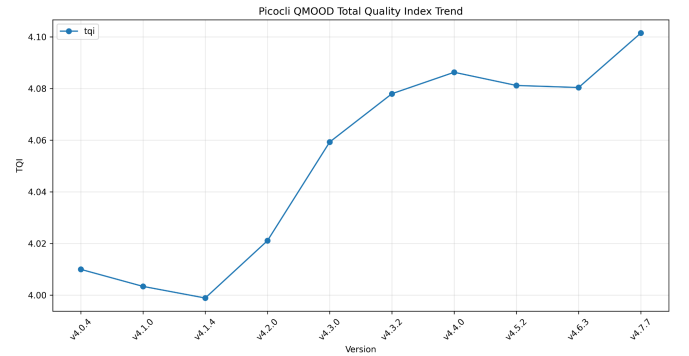


Fig. 7. QMOOD Total Quality Index trend.

TABLE IV
QMOOD QUALITY SCORES

Version	Reuse.	Flex.	Underst.	Func.	Extend.	Effect.	TQI
v4.0.4	1.0000	1.0000	-0.9900	1.0000	1.0000	1.0000	4.0100
v4.1.0	1.0063	1.0029	-1.0015	1.0032	0.9927	0.9998	4.0034
v4.1.4	1.0058	1.0024	-1.0040	1.0032	0.9916	0.9998	3.9989
v4.2.0	1.0319	1.0208	-1.0268	1.0152	0.9768	1.0033	4.0211
v4.3.0	1.0655	1.0458	-1.0593	1.0293	0.9684	1.0096	4.0593
v4.3.2	1.0705	1.0466	-1.0611	1.0314	0.9775	1.0131	4.0780
v4.4.0	1.0803	1.0564	-1.0910	1.0346	0.9864	1.0195	4.0863
v4.5.2	1.0809	1.0552	-1.0938	1.0354	0.9840	1.0195	4.0812
v4.6.3	1.0927	1.0911	-1.1419	1.0403	0.9688	1.0293	4.0804
v4.7.7	1.1132	1.1054	-1.1579	1.0498	0.9578	1.0332	4.1015

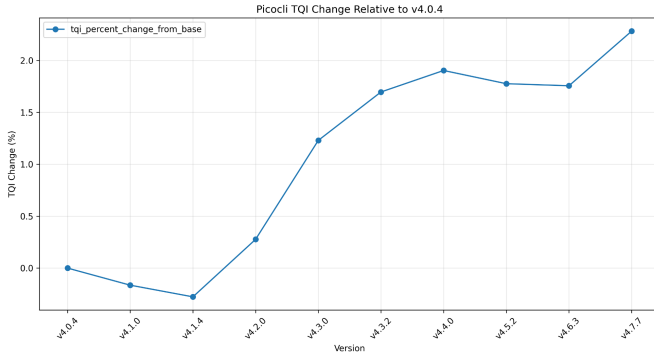


Fig. 8. TQI percentage change relative to v4.0.4.

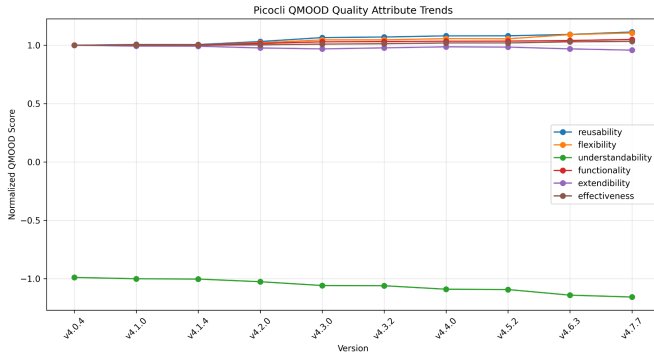


Fig. 9. QMOOD quality attribute trends. Since several attributes remain close to the normalized baseline, this figure should be interpreted together with Table IV.

makes the smaller differences among the remaining attributes easier to read. Figure 11 connects the QMOOD quality attributes to their underlying design properties.

C. Class-Level Risk Analysis

Table V lists the top ten risky classes in v4.7.7 according to Equation 8. The risk is not evenly distributed across the system. It is concentrated around `picocli.CommandLine` and its inner classes.

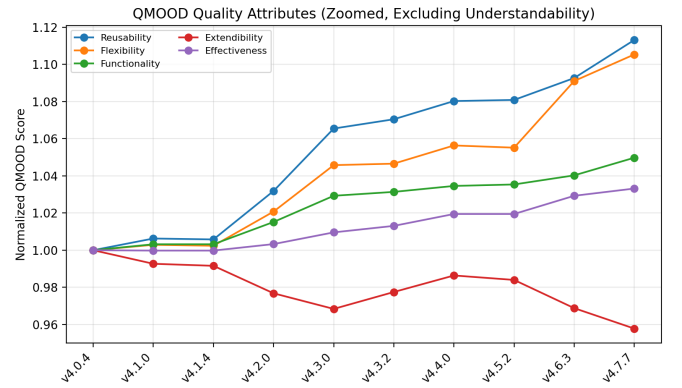


Fig. 10. Zoomed view of QMOOD attributes excluding understandability.

The highest-risk class, `picocli.CommandLine`, has simultaneously high WMC, RFC, LCOM, and CBO values. This makes it a strong candidate for focused design inspection and refactoring. The DIT value of 76 for this class should be treated cautiously because it may be affected by how CK Tool interprets nested and inner class structures.

D. LLM Evaluation Results

Tables VI and VII show the LLM comparison for Prompt 1 and Prompt 2. The six criteria are metric grounding, consistency, technical correctness, refactoring quality, hallucination risk control, and version-trend understanding.

Gemini produced strong metric-grounded interpretations but sometimes used design-smell terms such as God Class more confidently than the available evidence allowed. DeepSeek captured the general trend but made technical interpretation errors, particularly around DAM and abstraction. Claude produced the most balanced and academically cautious outputs, especially under Prompt 2. Overall, LLMs were useful for generating interpretive summaries and refactoring ideas, but their outputs required validation against the measured metrics.

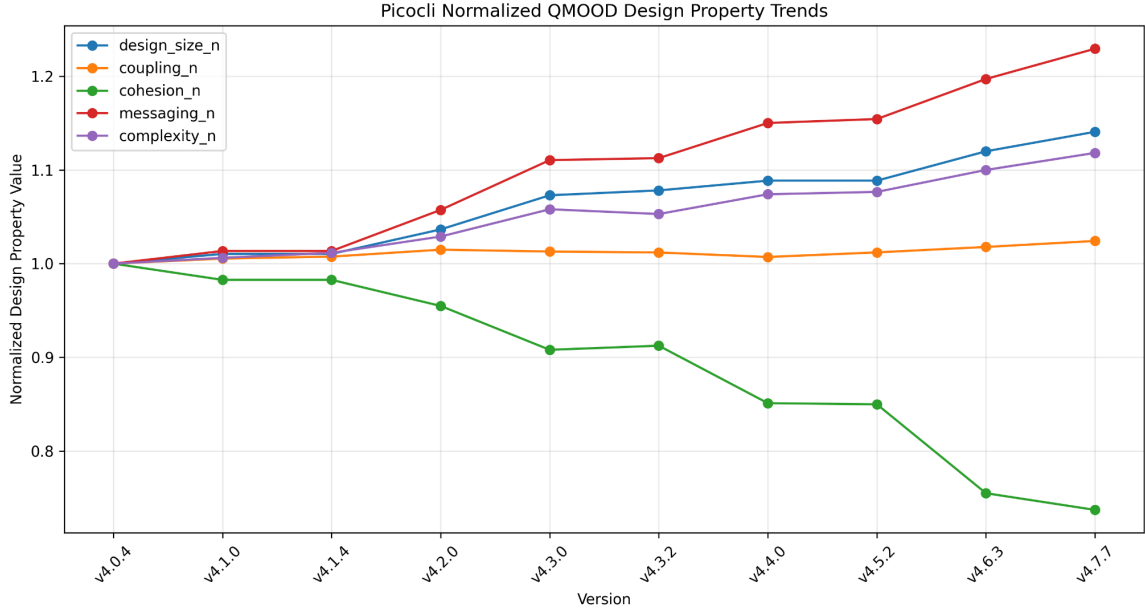


Fig. 11. Normalized QMOOD design properties across versions.

TABLE V
TOP RISKY CLASSES IN v4.7.7

Rank	Class	WMC	RFC	LCOM	CBO	DIT	Risk
1	picocli.CommandLine	362	338	17878	66	76	0.893
2	picocli.CommandLine\$Model\$CommandSpec	388	324	10708	50	1	0.674
3	picocli.CommandLine\$Interpreter	527	375	2032	70	1	0.640
4	picocli.CommandLine\$Help	207	245	2635	40	2	0.367
5	picocli.CommandLine\$Model\$ArgSpec	184	109	2090	31	1	0.253
6	picocli.CommandLine\$Model\$UsageMessageSpec	206	111	2809	14	1	0.242
7	picocli.CommandLine\$Model\$ArgGroupSpec	137	114	741	34	1	0.213
8	picocli.AutoComplete	156	132	778	24	1	0.211
9	picocli.CommandLine\$Model\$CommandReflection	101	135	153	35	1	0.198
10	picocli.CommandLine\$Model\$ArgSpec\$Builder	97	81	2215	27	1	0.190

TABLE VI
PROMPT 1 LLM EVALUATION RESULTS. REF. DENOTES REFACTORING QUALITY, HALL. DENOTES HALLUCINATION-RISK CONTROL, AND TREND DENOTES VERSION-TREND UNDERSTANDING.

Model	Metric	Cons.	Tech.	Ref.	Hall.	Trend	Total
Gemini	5	4	4	4	3	5	25/30
DeepSeek	5	3	3	3	3	5	22/30
Claude	5	5	4	4	4	5	27/30

TABLE VII
PROMPT 2 LLM EVALUATION RESULTS. REF. DENOTES REFACTORING QUALITY, HALL. DENOTES HALLUCINATION-RISK CONTROL, AND TREND DENOTES VERSION-TREND UNDERSTANDING.

Model	Metric	Cons.	Tech.	Ref.	Hall.	Trend	Total
Gemini	5	4	4	4	3	5	25/30
DeepSeek	5	3	3	3	1	5	20/30
Claude	5	5	4	4	4	5	27/30

VI. DISCUSSION

The results show that picoccli experienced controlled functional growth between v4.0.4 and v4.7.7. The slight TQI increase indicates that the system did not degrade globally according to the QMOOD model. However, the decrease in understandability and extendibility is important. These two attributes are affected by increasing design size, complexity, coupling, and polymorphism-related proxies. Therefore, the TQI increase should not be interpreted as unconditional improvement.

The DAM values remained stable and slightly improved, which means that encapsulation did not become worse at the field-access level. This is a positive result because the decline in understandability is more strongly associated with increasing WMC, RFC, and LCOM than with degraded data hiding. In other words, the main concern is not field visibility but the concentration of behavior and responsibility in large classes.

The LCOM trend and the class-level risk table support this interpretation. The project-level averages show increasing cohesion risk, while the v4.7.7 risk table shows that this risk is concentrated around `picoccli.CommandLine`. This finding suggests that refactoring should not be applied uniformly across the whole project. Instead, a focused inspection of high-risk classes would be more effective.

There are also limitations. First, QMOOD values are relative to the baseline version, not absolute quality values. Second, some QMOOD design properties are represented by proxy metrics because CK Tool does not directly provide every original QMOOD measure. Third, inner class handling can affect some CK measurements, as seen in the DIT outlier for `picoccli.CommandLine`. Fourth, LLM-based interpretation is not a substitute for code-level architectural analysis. The LLM outputs are useful as supporting qualitative evidence, but unsupported architectural claims must be rejected or treated as hypotheses.

VII. CONCLUSIONS AND FUTURE WORK

This study analyzed ten versions of picoccli using CK metrics, QMOOD quality attributes, class-level risk scoring, and LLM-assisted interpretation. The main finding is that picoccli grew from 192 to 219 classes and from 1517 to 1848 methods while its TQI increased slightly from 4.0100 to 4.1015. This indicates a limited positive overall quality trend. However, the underlying quality attributes reveal a trade-off: reusability, flexibility, functionality, and effectiveness improved, while understandability and extendibility declined.

The class-level risk analysis showed that the most important maintainability risks are concentrated around `picoccli.CommandLine` and its inner classes. Therefore, future refactoring work should focus on reducing WMC, RFC, and LCOM in these classes through Extract Class, Extract Method, responsibility separation, and interface simplification.

Future work can extend this study in three directions. First, the risky classes can be manually inspected and

refactored, followed by a repeated metric analysis to measure the effect of refactoring. Second, additional tools can be used to compute richer QMOOD metrics and reduce proxy dependence. Third, LLM evaluation can be expanded with more models, more prompt variants, and stricter evidence-grounding checks to reduce hallucination risk.

REFERENCES

- [1] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," *IEEE Transactions on Software Engineering*, vol. 20, no. 6, pp. 476–493, 1994.
- [2] W. Li and S. Henry, "Object-oriented metrics that predict maintainability," *Journal of Systems and Software*, vol. 23, no. 2, pp. 111–122, 1993.
- [3] J. Bansiya and C. G. Davis, "A hierarchical model for object-oriented design quality assessment," *IEEE Transactions on Software Engineering*, vol. 28, no. 1, pp. 4–17, 2002.
- [4] M. M. Lehman, "Programs, life cycles, and laws of software evolution," *Proceedings of the IEEE*, vol. 68, no. 9, pp. 1060–1076, 1980.
- [5] W. Cunningham, "The WyCash portfolio management system," in *Proc. ACM OOPSLA Experience Report*, 1992.
- [6] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley Professional, 2018.
- [7] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, "Large language models for software engineering: Survey and open problems," arXiv preprint arXiv:2310.03533, 2023.
- [8] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. C. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," arXiv preprint arXiv:2308.10620, 2023.
- [9] M. Aniche, "CK: Java code metrics calculator," GitHub repository. [Online]. Available: <https://github.com/mauricioaniche/ck>
- [10] R. Popma and contributors, "picoccli: A mighty tiny command line interface," GitHub repository. [Online]. Available: <https://github.com/remkop/picoccli>